



17 février 2026

Challenge BNP : Détection de Fraude

KI Alidou (alidou.ki@imt-atlantique.net)



IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Table des matières

1	Introduction	3
2	Analyse exploratoire des données	3
3	Approche méthodologique	6
4	Sélection et optimisation des modèles	8
5	Résultats	9
6	Limites	10
7	Conclusion	11

Liste des tableaux

2.	Analyse exploratoire des données	3
1	Taux de fraude par catégorie (item1) – principales catégories	3
2	Taux de fraude par fabricant – fabricants principaux	4
3	Taux de fraude par catégorie d’item – tous slots confondus	4
4	Taux de fraude selon le nombre de catégories uniques	4
5	Taux de fraude selon le nombre de fabricants uniques	5
6	Taux de fraude selon le prix maximum du panier	5
7	Statistiques comparées des variables quantitatives	6
4.	Sélection et optimisation des modèles	8
8	Hyperparamètres retenus après optimisation	8
5.	Résultats	9
9	Statistiques des probabilités prédites sur le test	9
10	Comparaison avec les benchmarks	10

1 Introduction

L'objectif de ce projet est de construire un modèle capable de prédire la probabilité qu'un achat financé par notre partenaire soit frauduleux. La base de données contient 115 988 observations décrivant le contenu de paniers d'achats, chaque panier pouvant contenir jusqu'à 24 items. Pour chaque item, on dispose de sa catégorie, son prix, son fabricant, la description du modèle, un code produit et la quantité achetée.

Le principal défi de ce problème est le fort déséquilibre des classes : seulement 1,4 % des observations correspondent à des fraudes (1 681 cas sur 115 988). La métrique retenue est l'aire sous la courbe Précision-Rappel (PR-AUC), qui est particulièrement adaptée pour évaluer un modèle sur la classe minoritaire dans ce type de contexte.

L'échantillon d'entraînement contient 92 790 observations (dont 1 319 fraudes) et l'échantillon de test 23 198 observations (dont 362 fraudes).

2 Analyse exploratoire des données

Avant de construire un modèle, j'ai cherché à comprendre les caractéristiques des transactions frauduleuses par rapport aux transactions normales. L'idée était d'identifier les variables et les profils les plus discriminants pour guider ensuite la création de features.

2.1 Taux de fraude par catégorie de produit

J'ai d'abord regardé le taux de fraude en fonction de la catégorie du premier item du panier (`item1`). Les résultats montrent des écarts importants entre catégories :

Table 1 – Taux de fraude par catégorie (`item1`) – principales catégories

Catégorie	Nb obs	Nb fraudes	Taux (%)
TELEPHONES, FAX MACHINES & TWO-WAY RADIOS	2 952	86	2.91
COMPUTERS	47 436	1 066	2.25
TELEPHONES FAX MACHINES TWO-WAY RADIOS	1 417	31	2.19
BABY CHILD TRAVEL	484	10	2.07
AUDIO ACCESSORIES	386	6	1.55

On observe que les catégories Téléphones et Computers concentrent la grande majorité des fraudes, à la fois en volume et en taux. C'est cohérent avec le fait que ces produits ont une valeur de revente élevée, ce qui les rend attractifs pour les fraudeurs. Certaines catégories comme Kitchen Utensils affichent des taux très élevés (33 %) mais sur des effectifs trop faibles (3 observations) pour être significatifs.

2.2 Taux de fraude par fabricant

En agrégeant les fabricants sur tous les slots (de `make1` à `make24`), on obtient une vision plus complète. Certains fabricants de niche présentent des taux de fraude élevés, mais les deux fabricants les plus représentés sont Apple et Samsung :

Table 2 – Taux de fraude par fabricant – fabricants principaux

Fabricant	Nb apparitions	Nb fraudes	Taux (%)
APPLE	76 509	1 391	1.82
SAMSUNG	5 084	86	1.69
BUGABOO	342	14	4.09
LE CREUSET	208	12	5.77
MAXI-COSI	125	9	7.20

Apple domine largement le jeu de données en volume et concentre une part très importante des fraudes. Des fabricants comme Maxi-Cosi ou Le Creuset ont des taux plus élevés mais sur des effectifs plus modestes.

2.3 Taux de fraude par item (tous slots confondus)

En regardant les catégories d'items sur l'ensemble des 24 positions du panier (et non seulement le premier item), on fait apparaître des catégories intéressantes :

Table 3 – Taux de fraude par catégorie d'item – tous slots confondus

Catégorie	Nb apparitions	Nb fraudes	Taux (%)
KITCHEN UTENSILS & GADGETS	31	11	35.48
STORAGE & ORGANISATION	39	8	20.51
AUDIO ACCESSORIES	1 670	82	4.91
TELEPHONES, FAX MACHINES...	3 138	88	2.80
COMPUTERS	50 221	1 152	2.29
FULFILMENT CHARGE	25 023	555	2.22

On remarque que Fulfilment Charge apparaît fréquemment dans les paniers frauduleux. C'est un frais de livraison qui accompagne souvent les achats de produits électroniques, ce qui confirme le profil de fraude orienté vers les produits technologiques à forte valeur.

2.4 Taux de fraude selon la diversité du panier

J'ai ensuite analysé le lien entre la diversité du panier et la fraude, en comptant le nombre de catégories uniques et le nombre de fabricants uniques par panier.

Table 4 – Taux de fraude selon le nombre de catégories uniques

Nb catégories uniques	Nb obs	Nb fraudes	Taux (%)
1	51 078	671	1.31
2	32 299	567	1.76
3	7 184	56	0.78
4	1 429	10	0.70
5	372	5	1.34
6	167	5	2.99

Table 5 – Taux de fraude selon le nombre de fabricants uniques

Nb fabricants uniques	Nb obs	Nb fraudes	Taux (%)
1	57 206	737	1.29
2	32 566	545	1.67
3	1 499	15	1.00
4	381	3	0.79
5	158	5	3.16

On constate que les paniers avec 2 catégories ou 2 fabricants différents ont un taux de fraude légèrement plus élevé que les paniers homogènes. Au-delà de 3 catégories, les effectifs deviennent trop faibles pour tirer des conclusions fiables. Cette observation suggère que les fraudeurs achètent souvent un produit principal accompagné d'un accessoire ou d'un frais de livraison.

2.5 Taux de fraude selon le prix le plus élevé du panier

Le prix de l'article le plus cher du panier est un indicateur important :

Table 6 – Taux de fraude selon le prix maximum du panier

Tranche de prix max	Nb obs	Nb fraudes	Taux (%)
100–300	459	8	1.74
300–500	13 738	38	0.28
500–800	14 030	120	0.86
800–1 000	20 350	225	1.11
1 000–1 500	25 302	459	1.81
1 500–2 000	10 576	263	2.49
2 000–3 000	7 110	186	2.62
3 000+	1 222	20	1.64

Le taux de fraude augmente nettement avec le prix maximum du panier, passant de 0.28 % pour les paniers dont l'article le plus cher coûte entre 300 et 500 € à 2.62 % pour la tranche 2 000–3 000 €. On observe cependant un léger recul pour les paniers au-delà de 3 000 €, probablement parce que ces transactions font l'objet de contrôles renforcés. Cela confirme que les fraudeurs ciblent les produits à forte valeur de revente.

2.6 Statistiques descriptives des variables quantitatives

Table 7 – Statistiques comparées des variables quantitatives

Variable	Groupe	Moyenne	Médiane	Écart-type	Min	Max
cash_price1	Non fraude	1 089	949	710	2	21 995
cash_price1	Fraude	1 401	1 199	742	8	6 999
prix_total	Non fraude	1 230	1 099	770	219	21 995
prix_total	Fraude	1 547	1 379	831	305	9 018
Nb_of_items	Non fraude	1.76	1.00	1.46	1	60
Nb_of_items	Fraude	1.76	2.00	1.55	1	24
nb_produits	Non fraude	1.85	1.00	1.98	1	98
nb_produits	Fraude	1.83	2.00	2.22	1	44

Les paniers frauduleux présentent un prix moyen du premier item significativement plus élevé (1 401 € contre 1 089 €) et un prix total du panier également supérieur (1 547 € contre 1 230 €). En revanche, le nombre d'items et le nombre de produits sont similaires entre les deux groupes, ce qui suggère que ce n'est pas la taille du panier qui distingue les fraudes mais plutôt la valeur unitaire des produits achetés. On note aussi que la médiane du nombre d'items est de 2 pour les fraudes contre 1 pour les non-fraudes, ce qui rejoint l'observation sur la diversité : les fraudeurs ajoutent souvent un second item (accessoire ou frais).

3 Approche méthodologique

3.1 Démarche générale

Ma démarche a suivi plusieurs étapes itératives :

1. Exploration des données pour comprendre la structure du problème et identifier les variables discriminantes.
2. Création de features (feature engineering) à partir des variables brutes pour enrichir l'information disponible.
3. Encodage des variables catégorielles par target encoding lissé pour transformer le texte en variables numériques exploitables.
4. Test de plusieurs familles de modèles (bagging et boosting) avec validation croisée stratifiée.
5. Optimisation des hyperparamètres par grid search sur les modèles de boosting retenus.
6. Combinaison des modèles (blending) avec ajustement manuel des poids.

3.2 Feature engineering

À partir des 144 variables brutes organisées en 24 slots, j'ai construit environ 35 nouvelles features réparties en plusieurs catégories :

- Agrégats de prix : prix total, prix moyen, prix maximum, prix minimum (hors zéros), écart-type des prix, et leurs transformations logarithmiques. Le logarithme permet de compresser les valeurs extrêmes et de stabiliser la distribution.
- Ratios de prix : part du prix du premier item dans le total, part du prix maximum dans le total. Ces ratios mesurent la concentration du panier sur un seul article.
- Agrégats de quantité : nombre total de produits, quantité maximale par item, et le log du nombre de produits.
- Ratios clés : prix par produit, prix par item, nombre de produits par item. Ces métriques capturent le prix unitaire moyen et le regroupement des produits.
- Indicateurs binaires : panier mono-item, présence d'un produit cher ($> 1\,500\text{€}$), catégorie à risque (Computers, Téléphones), fabricant Apple.
- Mesures de diversité : nombre de fabricants uniques, nombre de catégories uniques, ratio de diversité, indicateur « tous même fabricant ».
- Interactions : combinaisons comme Apple $\times$ produit cher, catégorie à risque $\times$ panier mono-item. Ces features permettent au modèle de capter des profils de fraude spécifiques.
- Combinaisons textuelles : concaténation de la catégorie et du fabricant (`item1_make1`), de deux catégories successives (`item1_item2`), etc. Ces combinaisons créent des profils plus fins qui sont ensuite encodés numériquement.

3.3 Encodage des variables catégorielles

Les variables textuelles (catégorie, fabricant, modèle, code produit) ont été transformées en variables numériques par target encoding lissé. Pour chaque valeur d'une variable catégorielle, on calcule le taux de fraude observé dans le groupe, puis on le lisse avec la moyenne globale pour éviter le surapprentissage sur les petits groupes :

$$\text{score}_{\text{lissé}} = \frac{n \times \bar{y}_{\text{groupe}} + m \times \bar{y}_{\text{global}}}{n + m} \quad (1)$$

où n est le nombre d'observations du groupe, $\bar{y}_{\text{groupe}}$ le taux de fraude du groupe, $\bar{y}_{\text{global}}$ le taux de fraude global, et m le paramètre de lissage (fixé à 20 pour les variables avec peu de modalités, 50 pour les variables à forte cardinalité comme `model1`).

Quand un groupe a beaucoup d'observations, le score tend vers le taux réel du groupe. Quand il en a peu, il est ramené vers la moyenne globale, ce qui est plus prudent.

En complément du target encoding, j'ai aussi calculé pour chaque modalité sa fréquence d'apparition, son nombre d'occurrences et sa déviation par rapport à la moyenne globale, ce qui donne 4 colonnes numériques par variable catégorielle encodée.

3.4 Gestion du déséquilibre des classes

Le taux de fraude de 1,4 % implique un fort déséquilibre. Pour y remédier, j'ai utilisé plusieurs mécanismes :

- Validation croisée stratifiée : chaque fold conserve la même proportion de fraudes que l'ensemble complet.

- Pondération des classes : dans XGBoost, le paramètre `scale_pos_weight` donne un poids de ~ 69 à chaque observation frauduleuse. Dans HistGBM, `class_weight='balanced'` effectue un rééquilibrage automatique équivalent.
- Métrique adaptée : l'optimisation et l'évaluation sont faites sur la PR-AUC, qui est conçue pour les classes déséquilibrées.

4 Sélection et optimisation des modèles

4.1 Exploration initiale

Dans une première phase, j'ai testé plusieurs familles de modèles en validation croisée stratifiée à 5 folds :

- Modèles de bagging : Random Forest, Extra Trees.
- Modèles de boosting : Gradient Boosting, XGBoost, HistGradientBoosting.

Les modèles de boosting ont systématiquement surpassé les modèles de bagging sur la PR-AUC. Ce résultat est attendu : le boosting construit séquentiellement des arbres qui corrigent les erreurs des précédents, ce qui le rend plus efficace pour détecter les patterns rares comme la fraude. Les modèles de bagging, qui construisent des arbres indépendants et les moyennent, ont plus de difficulté à se concentrer sur la classe minoritaire.

J'ai donc retenu les trois modèles de boosting pour la suite.

4.2 Optimisation des hyperparamètres

Pour chaque modèle, j'ai effectué une recherche par grille (grid search) en validation croisée stratifiée pour trouver les meilleurs hyperparamètres. Les paramètres principaux optimisés sont :

Table 8 – Hyperparamètres retenus après optimisation

Paramètre	GradientBoosting	XGBoost	HistGBM
Nombre d'arbres	800	526	800
Learning rate	0.03	0.02	0.02
Profondeur max	5	6	5
Min samples/feuille	50	50	50
Subsample	0.7	0.7	–
Colsample	–	0.6	–
Régularisation L1	–	0.5	–
Régularisation L2	–	2.0	2.0

Les learning rates faibles (0.02–0.03) combinés à un nombre élevé d'arbres permettent un apprentissage progressif et stable. Les valeurs de `min_samples_leaf` à 50 et les régularisations empêchent les arbres de s'adapter au bruit des données. Le sous-échantillonnage (`subsample` à 0.7 et `colsample_bytree` à 0.6) apporte de la diversité entre les arbres et réduit l'overfitting.

Pour des raisons de lisibilité, j'ai pas ajouté la partie où je testais les différents modèles ainsi que leurs optimisations dans le notebook final.

4.3 Combinaison des modèles (blending)

Plutôt que de choisir un seul modèle, j’ai combiné les prédictions des trois modèles par moyenne pondérée :

$$P_{\text{final}} = 0.25 \times P_{\text{GB}} + 0.10 \times P_{\text{XGBoost}} + 0.65 \times P_{\text{HistGBM}} \quad (2)$$

Le choix initial des poids a été guidé par les performances individuelles de chaque modèle en validation croisée : HistGBM avait la meilleure PR-AUC, d’où son poids plus élevé. J’ai ensuite ajusté manuellement les poids en testant plusieurs configurations et en soumettant les résultats, jusqu’à trouver la combinaison la plus performante.

L’intérêt du blending est que chaque modèle capte des patterns légèrement différents. Même si les trois sont des modèles de boosting, leurs implémentations diffèrent (algorithme d’optimisation, gestion des valeurs manquantes, méthode de régularisation), ce qui crée une complémentarité.

5 Résultats

5.1 Prédictions sur l’échantillon de test

Les trois modèles ont été entraînés sur l’ensemble des données d’entraînement puis appliqués à l’échantillon de test :

Table 9 – Statistiques des probabilités prédites sur le test

Modèle (poids)	Min	Max	Moyenne
GradientBoosting (0.25)	0.0001	0.9849	0.0130
XGBoost (0.10)	0.0002	0.9909	0.1996
HistGBM (0.65)	0.0004	0.9877	0.2251
Blend final	0.0004	0.9840	0.1695

On observe que les moyennes des probabilités prédites diffèrent fortement entre les modèles : GradientBoosting produit une moyenne proche du taux de fraude réel (1.3 %), tandis que XGBoost et HistGBM ont des moyennes plus élevées (20 % et 22 %). Cette différence vient du rééquilibrage des classes : `scale_pos_weight` et `class_weight='balanced'` gonflent les probabilités prédites pour mieux discriminer les fraudes. Cela n’affecte pas la qualité du classement (et donc la PR-AUC), car ce qui compte est l’ordre relatif des probabilités et non leur valeur absolue.

5.2 Performance finale

La soumission sur les données de test a donné un score de :

$$\boxed{\text{PR-AUC} = 0.1978} \quad (3)$$

Ce résultat est à comparer aux benchmarks fournis :

Table 10 – Comparaison avec les benchmarks

Méthode	PR-AUC	Amélioration
Benchmark 1 (aléatoire)	0.017	–
Benchmark 2 (ML optimisé)	0.140	–
Notre modèle	0.1978	+41 % vs Benchmark 2

Notre modèle dépasse le benchmark 2 de plus de 41 % en termes relatifs. Il est environ 11.6 fois meilleur qu’un modèle aléatoire.

5.3 Interprétation du score

Un PR-AUC de 0.1978 peut sembler faible en valeur absolue, mais il faut le replacer dans le contexte d’un taux de fraude de seulement 1.4 %. Un modèle aléatoire obtient un PR-AUC de 0.014 (environ le taux de fraude), ce qui montre que notre modèle fait nettement mieux que le hasard.

La PR-AUC mesure la capacité du modèle à classer les fraudes devant les non-fraudes dans l’ordre des probabilités prédites. Une valeur de 0.1978 signifie que le modèle identifie correctement une proportion significative de fraudes parmi les transactions jugées les plus suspectes, tout en maintenant un niveau de précision acceptable.

6 Limites

6.1 Overfitting du target encoding

L’encodage par la cible (target encoding) introduit un risque de fuite d’information : le taux de fraude d’un groupe est calculé sur les mêmes données qui servent à l’entraînement. J’ai tenté d’atténuer ce problème en utilisant un lissage important (paramètre m entre 20 et 50), mais un target encoding par validation croisée aurait pu réduire davantage cet effet.

6.2 Ajustement manuel des poids

Les poids du blending ont été ajustés manuellement par essais successifs. Une approche plus rigoureuse aurait consisté à utiliser une régression logistique ou une optimisation automatique sur les prédictions out-of-fold pour déterminer les poids optimaux. Néanmoins, l’ajustement manuel a permis d’atteindre un résultat satisfaisant.

7 Conclusion

Ce projet m'a permis de mettre en pratique une démarche complète de machine learning appliquée à la détection de fraude. L'analyse exploratoire a révélé que les fraudes ciblent principalement les produits électroniques de marque à forte valeur (Apple, Computers, Téléphones), avec des paniers dont le prix maximum se situe entre 1 000 et 3 000 €.

Le feature engineering a joué un rôle central en transformant les 144 variables brutes en features plus informatives : agrégats de prix, mesures de diversité, indicateurs de risque et combinaisons catégorielles. L'approche par blending de trois modèles de gradient boosting a permis de tirer parti de la complémentarité des algorithmes.

Le score final de PR-AUC = 0.1978 dépasse le benchmark de 41 %, ce qui valide l'approche adoptée. Des améliorations seraient possibles notamment par un target encoding par validation croisée, une optimisation automatique des poids du blend, ou l'ajout de features plus complexes.